

1. Prerequisites

Make sure you have the following installed:

- **Docker Desktop** (Windows/macOS) or **Docker Engine** (Linux)
- **Docker Compose v2** (included with Docker Desktop)
- **Python 3.10+**

Verify installation:

```
docker --version
docker compose version
python --version
```

2. Project layout

Create a new project directory named `kafka-lab`:

```
kafka-lab/
├── docker-compose.yml
├── scripts/           # (optional) helper shell scripts
├── app/               # local python code (producer/consumer)
│   ├── venv/          # created later
│   ├── requirements.txt
│   ├── producer.py
│   └── consumer.py
```

3. Docker Compose — Single Node Kafka

Create a file `docker-compose.yml` in the `kafka-lab` folder:

```
version: "3.8"
```

```
services:
```

```
  kafka:
```

```
    image: apache/kafka:latest
```

```
    container_name: kafka
```

```
    restart: unless-stopped
```

```
    ports:
```

```
      - "9092:9092"    # client access from your laptop
```

```
      - "9093:9093"    # controller listener (internal)
```

```

environment:

    # --- KRaft essentials ---

    KAFKA_PROCESS_ROLES: "broker,controller"           # process.roles

    KAFKA_NODE_ID: "1"                                # node.id

    KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:9093"     #
controller.quorum.voters

    # --- Listeners / protocols ---

    KAFKA_LISTENERS: "PLAINTEXT://:9092,CONTROLLER://:9093" #
listeners

    KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://localhost:9092" #
advertised.listeners

    KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"      #
controller.listener.names

    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
"PLAINTEXT:PLAINTEXT,CONTROLLER:PLAINTEXT"

    KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"

    # --- QoS for local dev ---

    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: "1"

    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: "1"

    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: "1"

    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"

    # Optional: pin a cluster id (otherwise the image auto-generates one)

    # KAFKA_CLUSTER_ID: "5L6g3nShT-eMCtK--X86sw"

volumes:

    - kafka_data:/var/lib/kafka/data

healthcheck:

```

```
test: ["CMD-SHELL", "/opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list >/dev/null 2>&1"]
```

```
interval: 10s
```

```
timeout: 5s
```

```
retries: 8
```

```
volumes:
```

```
kafka_data:
```

Start Kafka:

```
cd kafka-lab
docker compose up -d
docker compose ps
```

Wait until the container is marked as **healthy**.

To see logs:

```
docker compose logs -f
```

4. Create a Topic

We'll use a topic named `demo.topic`.

Create the topic:

```
docker exec -it kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --create --topic demo.topic --partitions 1 --replication-factor 1
```

Verify it exists:

```
docker exec -it kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 --list
```

5. Python Virtual Environment

Inside the `app/` directory, set up your Python environment.

```
cd app

# Create and activate virtual environment
# macOS/Linux:
python -m venv venv
```

```
source venv/bin/activate

# Windows PowerShell:
# python -m venv venv
# .\venv\Scripts\Activate.ps1
```

Install dependencies:

```
echo "kafka-python==2.0.2" > requirements.txt
pip install -r requirements.txt
```



We use **kafka-python** for simplicity.

For production, consider **confluent-kafka** (faster, requires native `librdkafka`).

6. Python Producer

Create a file **producer.py** in the `app/` folder:

```
"""
Simple Kafka producer using kafka-python.
Sends demo messages to topic 'demo.topic'.

Run:  python producer.py
"""

from kafka import KafkaProducer
from kafka.errors import NoBrokersAvailable
import json, time, uuid

TOPIC = "demo.topic"
BOOTSTRAP = "localhost:9092"

def main():
    try:
        producer = KafkaProducer(
            bootstrap_servers=BOOTSTRAP,
            acks="all",          # wait for full commit
            linger_ms=5,         # short delay to batch messages
            value_serializer=lambda v: json.dumps(v).encode("utf-8"),
            key_serializer=lambda k: str(k).encode("utf-8") if k else None,
        )
    except NoBrokersAvailable:
        print(f"ERROR: Cannot connect to Kafka at {BOOTSTRAP}. Is Docker running?")
        return

    print(f"Producing to topic '{TOPIC}'...")
    for i in range(1, 6):
        event = {
            "id": str(uuid.uuid4()),
            "type": "demo",
            "seq": i,
            "payload": {"msg": f"hello #{i}"}
        }
        key = f"user-{i%2}" # optional message key
```

```

        future = producer.send(TOPIC, value=event, key=key)
        record_md = future.get(timeout=10)
        print(f"--> sent to partition {record_md.partition}, offset
{record_md.offset}")
        time.sleep(0.5)

    producer.flush()
    producer.close()
    print("Done.")

if __name__ == "__main__":
    main()

```

7. Python Consumer

Create a file **consumer.py** in the same folder:

```

"""
Simple Kafka consumer using kafka-python.
Reads messages from 'demo.topic'.

Run: python consumer.py
Stop: CTRL+C
"""

from kafka import KafkaConsumer
import json

TOPIC = "demo.topic"
BOOTSTRAP = "localhost:9092"
GROUP_ID = "demo.group"

def main():
    consumer = KafkaConsumer(
        TOPIC,
        bootstrap_servers=BOOTSTRAP,
        group_id=GROUP_ID,
        enable_auto_commit=True,
        auto_offset_reset="earliest",
        value_deserializer=lambda b: json.loads(b.decode("utf-8")),
        key_deserializer=lambda b: b.decode("utf-8") if b else None,
    )

    print(f"Consuming from topic '{TOPIC}' as group '{GROUP_ID}'...")
    try:
        for msg in consumer:
            print(f"[partition={msg.partition} offset={msg.offset}]
key={msg.key} value={msg.value}")
    except KeyboardInterrupt:
        print("\nStopping consumer...")
    finally:
        consumer.close()

if __name__ == "__main__":
    main()

```

8. Run the Demo

Open **two** terminals.

Terminal A — Consumer:

```
cd kafka-lab/app
source venv/bin/activate    # or .\venv\Scripts\Activate.ps1 on Windows
python consumer.py
```

Terminal B — Producer:

```
cd kafka-lab/app
source venv/bin/activate
python producer.py
```

You should see messages appear in the **consumer** terminal, showing each record's key, partition, and offset.